

# Agile-SOFL Grammar

Author: Shaoying Liu, SEI, ECNU, China

Time: 23 May 2026

/\*The name of this file is: [Agile-SOFLGrammarForParser\\_LiuFinal.doc](#). In this file, the final abstract grammar of the Agile-SOFL language for writing structured hybrid specifications is defined. This grammar is suitable for building a parser for Agile-SOFL in a software tool to support the use of Agile-SOFL in writing hybrid specifications. A hybrid specification is a mixture of semi-formal process specifications and formal process specifications in a top-down structure. All the bottom level processes are defined using a formal Functional Scenario Form (FSF) style specification (Formal FSF-Specification) and all the non-bottom level processes are specified in a semi-formal Functional Scenario Form style specification (Semi-Formal FSF Specification).

This file is created by reviewing and modifying the existing grammar of the SOFL specification language defined in 2011 whose basis is the abstract SOFL grammar defined in the SOFL book: “Formal Engineering for Industrial Software Development using the SOFL Method”, which was published by Springer-Verlag in 2004. \*/

/\*----- Grammar of the Agile-SOFL Specification Language -----\*/

**Specification ::=**

**Modules ["."]**

/\*Making period “.” and “;” optional facilitates the writing of the  
parser for a single module, as described below. \*/

/\*----- Modules -----\*/

**Modules ::=**

**Top\_module {";" Module}**

**Top\_module ::=**

**"module" "SYSTEM\_Identifier ";"**

**Module\_body**

**"end\_module"**

/\*The above grammar rules are used to build a module hierarchy. When a single module is saved, it will be saved into both a “text” file and a “XML” file automatically. The following grammar rules are used to build a “Module Parser (MP)”. \*/

/\* -----Module-----\*/

Module ::=

    "module" Module-name [ "/" ( Parent-module-name) ";"  
        Module\_body  
    "end\_module"

Module-name ::= Identifier

Parent-module-name ::= Identifier

Module\_body ::=

    [ Const\_declaration ";" ]  
    [ Type\_declaration ";" ]  
    [ Var\_declaration ";" ]  
    [ Inv\_definition ";" ]  
    Process\_function\_specifications

Const\_declaration ::=

    "const" Identifier "=" Constant  
    { ";" Identifier "=" Constant }

Type\_declaration ::=

    "type" Identifier [ "/" Module\_or\_field\_access ] "=" Type  
    { ";" Identifier [ "/" Module\_or\_field\_access ] "=" Type }

/\*The "type" Identifier [ "/" Module\_or\_field\_access ] is designed to allow a newly declared composite type to inherit from an existing composite type defined in another module or the same module\*/

Module\_or\_field\_access ::=

    [Module\_name "."] Identifier

/\***Agile-SOFL rule:** A variable declared in a module can be used (read only) in its any sub-modules, but can be written only by processes defined in the same module for information hiding. This characteristic is the result of combination of OO language class

and the traditional structured programming languages. This characteristic is unique for Agile-SOFL \*/

```
Var_declaration ::=
    "var" Variable ":" Type
    {";" Variable ":" Type}
```

```
Inv_definition ::=
    "inv" Predicate_expression
    {";" Predicate_expression}
```

```
Variable ::=
    "ext" Identifier
    | "ext" "#" Identifier
    | Identifier
```

```
Process_function_specifications ::=
    Processes [;" Functions]
```

/\*The only difference between a process and a function is that a process may use and change external variables defined in the same module while a function is not allowed to do so. A function can only yield its result based on its parameters (i.e., input variables) \*/

```
Processes ::= Process {";" Process}
```

```
Functions ::=
    Function {";" Function}
```

```
Process ::=
    "process" ("Init" | Identifier) "(" [Parameter_declarations] ")"
    [Parameter_declarations]
    Process_body
    "end_process"
    | "process" Identifier "equal" Module_field_access
    "end_process"
```

/\*A process allows multiple output variables, but when a process is called in a specification, its returned value will be treated as a composite-type value with multiple field variables. In Agile-SOFL, we allow each output variable to be used independently as an individual variable rather than as an explicitly expressed field variable of the output composite-type value. This is just to provide convenience for use, which improves the OO languages. Furthermore, the second selectable rule states that a process can be “equal” to another process definition in either the same module or another module.\*/

Parameter\_declarations ::=

Identifier {" , " Identifier} ":" Type  
 | Parameter\_declarations {" , " Parameter\_declarations}

Process\_body ::=

[Ext]  
 FSF: FSF-specification  
 [Decomposition]  
 [Comment]

Ext ::=

"ext" Ext\_variables

Ext\_variables ::=

Ext\_variable [":" Type]  
 {Space Ext\_variable [":" Type]}

Ext\_variable ::=

"rd" Identifier  
 | "wr" Identifier

Space ::= " "

/\*An FSF (Functional Scenario Form) is a conjunction of functional scenarios. Each functional scenario is a conjunction of atomic predicates or their negations. A FSF specification is an expression similar to FSF but differs from it in the sense that a test condition expression in an FSF expression is not a conjunction of atomic predicate expression but a disjunctive normal form. \*/

```

FSF-specification ::=
FSF-expression |
FSF-expression " | " "others &&" Defining-condition-expression |
"others &&" Defining-condition-expression

```

```

FSF-expression ::=
Test-condition-expression "&&" Defining-condition-expression
{" | " Test-condition-expression "&&" Defining-condition-expression}

```

/\*A test condition is basically a constraint on the input variables and external variables\*/

```

Test-condition-expression ::= Disjunctive-normal-form

```

```

Defining-condition-expression ::= Disjunctive-normal-form

```

/\*A defining condition is basically a constraint on the output variables and wr type external variables\*/

```

Defining-condition ::= Disjunctive-normal-form

```

```

Disjunctive-normal-form ::= Conjunction {"or" Conjunction}

```

```

Conjunction ::= Atomic-predicate-expression {"and" Atomic-predicate-expression}

```

```

Atomic-predicate-expression ::=
Text
| Boolean_type_value
| General_expression
| Boolean_type_apply
| Relational_expression
| "forall" "[" Binding_list "]" {Quantifier_list} " | " Disjunctive-normal-form
| "exists" "[" Binding_list "]" {Quantifier_list} " | " Disjunctive-normal-form
| "(" Atomic-predicate-expression ")"

```

| "not" Atomic-predicate-expression

/\* The non-terminal **Text** is defined at the end of this file \*/

/\*Atomic-predicate-expression is defined later in this file. It can be an informal text or a formal relation\*/

Decomposition ::=

    "decom" Identifier

Comment ::=

    "comment" Text

/\*----- Function -----\*/

Function ::=

    "function" Identifier "(" [Parameter\_declarations] ")" ":" Type

        [FSF-specification]

        ["==" (Expression | "undefined")]

    "end\_function"

/\*----- Type -----\*/

Type ::=

    Type\_identifier

    |     "{ " Enumeration\_value { "," Enumeration\_value } "}"

    |     Other\_type

Type\_identifier ::=

    Module\_or\_field\_access

/\*

    A type identifier may denote a type identifier defined in the same module or another module or class.

\*/

Enumeration\_value ::=

    "<" String\_of\_characters ">"

Other\_type ::=

- Basic\_type
- | Set\_type
- | Sequence\_type
- | Composite\_type
- | Product\_type
- | Map\_type
- | Union\_type

Basic\_type ::=

"nat0" | "nat" | "int" | "real" | "char" | "string" |  
"bool" | "given"

Set\_type ::=

"set" "of" Type

Sequence\_type ::=

"seq" "of" Type

Composite\_type ::=

"composed" "of"  
Field\_list  
"end"

Product\_type ::=

Type "\*" Type {"\*" Type}

Map\_type ::=

"map" Type "to" Type

Field\_list ::=

Identifier ":" Type

{Identifier ":" Type}

Union\_type ::=

          "universal"  
      |       Type " | " Type { " | " Type }

/\*----- Constant -----\*/

Constant ::=

      | Basic\_type\_constant  
      | Set\_type\_constant  
      | Sequence\_type\_constant  
      | Map\_type\_constant  
      | Composite\_or\_product\_type\_constant

Basic\_type\_constant ::=

          "nil"  
      |       Boolean\_type\_value  
      |       Constant\_identifier  
      |       Number  
      |       Character\_value  
      |       Enumeration\_value

Boolean\_type\_value ::=

      "true" | "false"

Constant\_identifier ::=

      Module\_or\_field\_access

/\*

  The Constant\_identifier can be declared in the same module or another module

\*/

Number ::=

          Integer  
      |       Integer "." Digits



Integer ::=

        "-" Digits  
        |        Digits

Digits ::=

        Digit {Digit}

Digit ::=

        0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9

Character\_value ::=

        "" Character ""

Set\_type\_constant ::=

        "{ [Constant\_list] }"  
        |        "{ Integer ", "..." ", Integer }"

Constant\_list ::=

        Constant {", Constant}

Sequence\_type\_constant ::=

        "[ [Constant\_list] ]"  
        |        "[ Integer ", "..." ", Integer ]"  
        |        String\_value

String\_value ::=

        "" String\_of\_characters ""

String\_of\_characters ::=

        Character {Character}

Map\_type\_constant ::=

        "{ " -> " }"  
        |  
        "{ Constant " -> " Constant

`{" Constant "-> Constant} "}`

`Composite_or_product_type_constant ::=`

`"mk_"Module_or_field_access "(" Constant_list ")"`

`/*----- Relational_expression -----*/`

`/*`

This part is the most important and essential part of the grammar rules because it is used almost everywhere in an Agile-SOFL FSF specification. It is also because it provides necessary information for various kinds of verification tools as well, such as inspection, testing, and static analyzer. That is why we highlight the title of this part in red colour. \*/

`Boolean_type_apply ::=`

`"bound" "(" Identifier ")"`

`| "subset" "(" Set_type_expression "," Set_type_expression ")"`

`| "psubset" "(" Set_type_expression "," Set_type_expression ")"`

`| "is_"Type_name "(" Expression ")"`

`/*`

`is_type` is the function to decide type of the expression (SOFL book, p.205 12.3)

`*/`

`Type_name ::=`

`Type_identifier`

`| "nat0" | "nat" | "int" | "real" | "char" | "string"`

`| "bool" | "given" | "universal" | "sign"`

`Relational_expression ::=`

`Expression`

`("=" | "<>")`

`Expression`

`| Number_type_expression`

`("<" | "<=")`

`Number_type_expression`

```

        [("<" | "<=")
        Number_type_expression]
    |
        Number_type_expression
        (">" | ">=")
        Number_type_expression
        [(">" | ">=")
        Number_type_expression]
    |
        Expression "inset" Set_type_expression
    |
        Expression "notin" Set_type_expression
/*
    Refer to p.145 in the SOFL book.
*/

```

```

Quantifier_list ::=
    "forall" "[" Binding_list "]" {Quantifier_list}
    |
    "exists" ["!"] "[" Binding_list "]" {Quantifier_list}

```

```

Binding_list ::=
    Identifier_list ":" (Type_expression | Set_type_expression)
    {"," Identifier_list ":" (Type_expression | Set_type_expression)}

```

```

Identifier_list ::=
    Identifier {"," Identifier}

```

```

Number_type_expression ::=
    Number_expression
    |
    General_expression

```

```

Set_type_expression ::=
    Set_expression
    |
    General_expression

```

```

Sequence_type_expression ::=
    Sequence_expression
    |
    General_expression

```

Map\_type\_expression ::=

Map\_expression  
| General\_expression

Composite\_type\_expression ::=

Composite\_expression  
| Composite\_or\_product\_expression  
| General\_expression

Product\_type\_expression ::=

Product\_expression  
| Composite\_or\_product\_expression  
| General\_expression

Field\_in\_expression ::=

Composite\_expression  
| Composite\_or\_product\_expression  
| General\_expression

/\*

Possible Expression that contains field variables.

\*/

Type\_expression ::=

Module\_or\_field\_access  
| Other\_type  
| Set\_expression

/\*

Expression that shows type identifier or an expression that can be used as a type.

\*/

Apply\_expression ::=

Sequence\_expression  
| Map\_expression  
| Product\_expression  
| Composite\_or\_product\_expression  
| General\_expression

```
/*
```

This rule defines the expression before encountering “(“ that covers the following situations: (1) process call, (2) array element access, (3) map application, and (4) product type field application (SOFL book, p.169 and p. 185.)

For example:

```
    process (arg1)
```

```
*/
```

```
/*----- General_expression -----*/
```

```
/* General_expression describes all of possible expressions that yield a boolean value as  
the result.*/
```

```
General_expression ::=
```

```
    "nil"
```

```
    | Simple_variable
```

```
    | Field_access
```

```
    | General_type_apply
```

```
    | Compound_expression
```

```
    | "(" General_expression ")"
```

```
Simple_variable ::=
```

```
    ["~"] Identifier
```

```
Field_access ::=
```

```
    Field_in_expression "." Identifier
```

```
General_type_apply ::=
```

```
    Process_function_sequence_map_or_product_apply
```

```
    | "get" "(" Set_type_expression ")"
```

```
    | "hd" "(" Sequence_type_expression ")"
```

```
/*Note that the reason why get(set) and hd(seq) can be a predicate because the set and  
seq here can be a set or sequence of boolean expressions or constants. */
```

```
Process_function_sequence_map_or_product_apply ::=
```

```
    Apply_expression "(" [Arguments] ")"
```

```
/*
```

This rule covers (1) process call, (2) array element access like a[1], (3) map application,

(4) product type variable accesses to its field, such as `b(2)` where `b` is a product type variable and `2` is the index of the second position of a tuple in the product type. The following is a more complex example: if we define

```
Composite1 =
    composed of
        field1: Type1
        map1: map Type2 to Type3
    end
sequence1: seq of Composite1,
```

then we can have the expressions:

```
sequence1(3).map1(arg)
```

The syntactical analysis of this expression is as follows:

```
sequence
  ↓
Apply_expression(3)
  ↓
Field_in_expression . map1
  ↓
Apply_expression(arg)
  ↓
General_expression
```

\*/

Compound\_expression ::=

```
    If_expression
  |    Let_expression
  |    Case_expression
```

If\_expression ::=

```
"if" Predicate_expression
"then" Expression
"else" Expression
```

Let\_expression ::=

    "let" Pattern\_definition  
    "in" Expression

Pattern\_definition ::=

    Pattern\_equal\_definition\_list  
    |      Pattern\_binding

Pattern\_equal\_definition\_list ::=

    Identifier "=" Expression  
    {" , " Identifier "=" Expression }

Pattern\_binding ::=

    Identifier ":" Type\_expression [ " | " Predicate\_expression ]

Case\_expression ::=

    "case" Expression "of"  
        Implicit\_case\_alternatives  
        [" ; " Implicit\_default\_expression]  
    "end\_case"

Implicit\_case\_alternatives ::=

    Implicit\_case\_alternative { " ; " Implicit\_case\_alternative }

Implicit\_case\_alternative ::=

    Case\_patterns "->" Expression

Implicit\_default\_expression ::=

    "default" "->" Expression

/\*-----Expression-----\*/

Expression ::= Ordinary-expression |  
              "(" Expression ")"

Ordinary-expression ::=

    Number\_expression

- | Character\_expression
- | Enumeration\_expression
- | Set\_expression
- | Sequence\_expression
- | Map\_expression
- | Composite\_expression
- | Product\_expression
- | Composite\_or\_product\_expression
- | General\_expression

/\*----- **Number\_expressions** -----\*/

/\* This kind of expression involves variables of numeric types such as int, real, nat0, nat.  
\*/

Number\_expression ::=

- Sub\_number\_expression1 [("+" | "-") Sub\_number\_expression1]
- | "(" Number\_expression ")"

Sub\_number\_expression1 ::=

- Sub\_number\_expression2
- [("\*" | "/" | "div" | "rem" | "mod")
- Sub\_number\_expression2]

Sub\_number\_expression2 ::=

- Sub\_number\_expression3 ["\*\*" Sub\_number\_expression3]

Sub\_number\_expression3 ::=

- Number
- | Unary\_expression
- | Number\_type\_apply
- | Number\_type\_expression

Unary\_expression ::=

- "-" Number\_type\_expression



Number\_type\_apply ::=

- "abs" "(" Number\_type\_expression ")"
- | "floor" "(" Number\_type\_expression ")"
- | "card" "(" Set\_type\_expression ")"
- | "len" "(" Sequence\_type\_expression ")"

/\*----- Character\_expression -----\*/

/\* This kind of Expression yields a character as its result. \*/

Character\_expression ::=

- Character\_value
- | "(" Character\_expression ")"

/\*----- Enumeration\_expression -----\*/

/\* This kind of Expression yields a value of an Enumeration type. \*/

Enumeration\_expression ::=

- Enumeration\_value
- | "(" Enumeration\_expression ")"

/\*----- Set\_expression -----\*/

/\* This kind of Expression yields a value of a Set type.\*/

Set\_expression ::=

- Set\_constructor
- | Set\_type\_apply
- | "(" Set\_expression ")"

Set\_constructor ::=

- "{" "}"
- | "{" Expression\_list "}"
- | "{" Number\_type\_expression "," "... " "," Number\_type\_expression "}"
- | "{" Expression " | " Predicate\_expression "}"
- | "{" Expression " | " Binding\_list "&" Predicate\_expression "}"

Expression\_list ::=

Expression {" , " Expression }

Set\_type\_apply ::=

```

        "union" "(" Set_type_expression "," Set_type_expression ")"
    |
        "inter" "(" Set_type_expression "," Set_type_expression ")"
    |
        "diff" "(" Set_type_expression "," Set_type_expression ")"
    |
        "dunion" "(" Set_type_expression ")"
    |
        "dinter" "(" Set_type_expression ")"
    |
        "power" "(" Set_type_expression ")"
    |
        "elems" "(" Sequence_type_expression ")"
    |
        "inds" "(" Sequence_type_expression ")"
    |
        "dom" "(" Map_type_expression ")"
    |
        "rng" "(" Map_type_expression ")"

/*----- Sequence_expression -----*/
/* This kind of Expression yields a value of a Sequence type. */
Sequence_expression ::=
    Sequence_constructor
    |
    Sequence_type_apply
    |
    String_value
    |
    "(" Sequence_expression ")"
Sequence_constructor ::=
    "[" "]"
    |
    "[" Expression_list "]"
    |
    "[" Number_type_expression "," "..."," Number_type_expression "]"
    |
    "[" Expression "|" Predicate_expression "]"
    |
    "[" Expression "|" Binding_list "&" Predicate_expression "]"

Sequence_type_apply ::=
    "tl" "(" Sequence_type_expression ")"
    |
    "conc" "(" Sequence_type_expression "," Sequence_type_expression ")"
    |
    "dconc" "(" Sequence_type_expression ")"

/*----- Map_expression -----*/
/* This kind of Expression yields a value of a Map type.*/
Map_expression ::=
    Map_constructor
    |
    Map_type_apply
    |
    "(" Map_expression ")"

```

Map\_constructor ::=

```
        "{" "->" "}"
    |      "{" Expression "->" Expression
        {" "," Expression "->" Expression } "}"
    |      "{" Expression "->" Expression " | " Predicate_expression "}"
    |      "{" Expression "->" Expression " | "
        Binding_list "&" Predicate_expression "}"
```

Map\_type\_apply ::=

```
        "domrt" "(" Set_type_expression "," Map_type_expression ")"
    |      "rngrt" "(" Map_type_expression "," Set_type_expression ")"
    |      "domrb" "(" Set_type_expression "," Map_type_expression ")"
    |      "rngrb" "(" Map_type_expression "," Set_type_expression ")"
    |      "override" "(" Map_type_expression "," Map_type_expression ")"
    |      "inverse" "(" Map_type_expression ")"
    |      "comp" "(" Map_type_expression "," Map_type_expression ")"
```

/\*----- Composite\_or\_product\_expression -----\*/

/\* This kind of Expression yields a value of either a Composite type or a Product type. \*/

Composite\_or\_product\_expression ::=

```
        "mk_" Module_or_class_field_access "(" Expression_list ")"
    |      "(" Composite_or_product_expression ")"
```

/\*

"mk\_" is the Constructor for Composite or Product type in the following rules.

\*/

/\*----- Composite\_expression -----\*/

/\* This kind of Expression yields a value of a Composite type. \*/

Composite\_expression ::=

```
        Composite_type_apply
    |      "(" Composite_expression ")"
```

Composite\_type\_apply ::=

```
        "modify" "(" Composite_type_expression "," Modifying_field_list ")"
```

Modifying\_field\_list ::=

```
        Identifier "->" Expression {" "," Identifier "->" Expression }
```

```

/*-----Product_expression -----*/
/* This kind of Expression yields a value of a Product type. */
Product_expression ::=
    Product_type_apply
    | "(" Product_expression ")"

Product_type_apply ::=
    "modify" "(" Product_type_expression "," Modifying_value_list ")"

Modifying_value_list ::=
    Index "->" Expression {" Index "->" Expression}

Index ::=
    Digits

/*-----Identifier -----*/
Identifier ::=
    Letter {(Letter | Digit | "_")}

Letter ::=
    "a".."z"
    | "A".."Z"

Character ::=
    Letter
    | Digit
    | New_line
    | White_space
    | Other_characters

Other_characters ::=
    "!" | "\"" | "#" | "$" | "%" | "&" | "'" | "(" | ")" | "-" | "=" | "^" | "~" | "\x" |
    "|" | "@" | "`" | "[" | "{" | ";" | "+" | ":" | "*" | "}" | "]" | "," | "<" | "." | ">" |
    "/" | "?" | "_"

```

```
Brief_comment ::=
    "/*" Text "*/"
```

```
Text ::=
    String_of_characters
```

```
/*The following is an example of the whole Agile-SOFL specification */
```

```
module SYSTEM_Banking;
const
    withdraw_limit = 10000;
    transfer_limit = 200000;

type
    Customers = set of Member;
    A = seq of Customers;

var
    salary: nat;
    customers: Customers;

inv
    forall[c: customers] | c.deposit > 0;

process A (x,y: int) q1: nat, q2 : int, q3 : int
ext rd a : int
    wr b : nat
FSF :
    x > y && q1 > q2 + q3 ||
    x = y && q1 > q2 * q3 ||
    others && q1 = q2 + q3
decom: Banking_Decom
comment: ...
end_process ;
```

```
process B(...) ...
```

```
ext ...
```

```
FSF :
```

```
...
```

```
end_process ;
```

```
end_module ;
```

```
module Banking_Decom / SYSTEM_Banking;
```

```
...
```

```
end_module;
```

```
...
```

```
module D / Banking_Decom;
```

```
...
```

```
end_module. /*This is the last module in the whole specification, which is marked using  
the period mark . */
```